CSED211 컴퓨터 시스템 2026 가을 · 중간고사 10/29 (목) ·

Lecture 3 · Floating Point

9/22 (화) · 슬라이드 "Lecture 3 Floating Point" + 수업 녹취 정리 · **Data Lab 2의 직접 기반**

1 Fractional Binary Numbers

Binary Point 표현

- fractional binary number: binary point 오른쪽 비트가 2의 음의 거듭제곱을 나타내는 표현
  - 10진수와 같은 원리: 왼쪽 부분은 정수와 동일, 오른쪽은 $2^{-1}, 2^{-2}, 2^{-3}, \ldots$

$$b_m b_{m-1} \ldots b_1 b_0 \,.\, b_{-1} b_{-2} \ldots b_{-n} \;=\; \sum_{i=-n}^{m} b_i \, 2^i$$

> 예: $101.11_2 = 5\frac{3}{4}$ · 한 칸 오른쪽 shift = ÷2, 한 칸 왼쪽 shift = ×2

- $0.111111\ldots_2 = 1.0 - \epsilon$ → 1보다 아주 조금 작음, $1.0 - \epsilon$으로 표기
- $101.11_2 = 5\frac{3}{4}$: 5 = 101, 3/4 = 1/2 + 1/4 · $10.111_2 = 2\frac{7}{8}$ (2로 나눔 → 오른쪽 shift 1) · $1.0111_2 = 1\frac{7}{16}$ (다시 2로 나눔)
- 분모가 2^k이면 소수점 아래 k번째 비트까지 필요, 분자가 그 사이 비트 패턴을 결정
  - 예: $7/8$ → 분모 2^3 → 소수점 아래 3비트, 분자 7 = 111 → 0.111_2

표현의 한계

- **한계 ①**: $x/2^k$ 꼴의 유리수만 정확히 표현 가능. 나머지는 비트가 무한히 반복:

  ① $1/3 = 0.0101010101[01]\ldots_2 \cdot 1/5 = 0.001100110011[0011]\ldots_2 \cdot 1/10 = 0.0001100110011[0011]\ldots_2$

- **한계 ②**: w 비트 안에서 binary point 위치가 하나로 고정 → 범위가 제한됨. point를 오른쪽에 두면 큰 수는 되지만 정밀도 손실, 왼쪽에 두면 반대 → 두 마리 토끼를 못 잡음 → 범용 수치 표현에는 부동소수점이 유리 (고정 소수점은 임베디드·DSP 등에서는 사용)

2 IEEE Floating-Point Standard

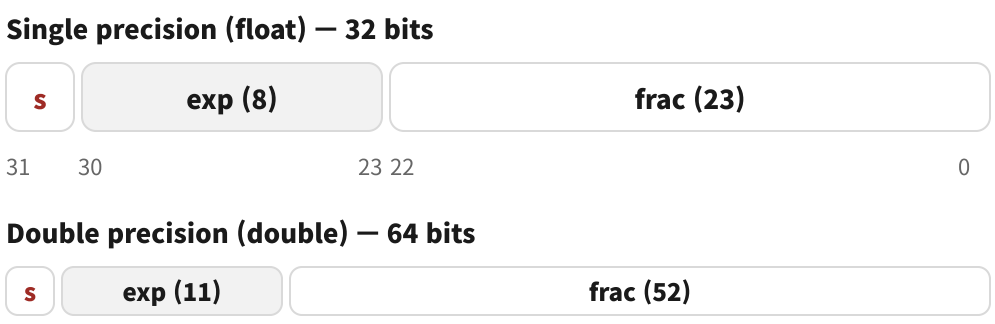
배경

- **IEEE 754**: 1985년 제정된 부동소수점 연산의 통일 표준. 이전에는 제조사마다 제각각이었으나 현재 모든 주요 CPU가 지원
- 수치 해석 관점이 주도 → rounding, overflow, underflow, $\pm\infty$, NaN 처리를 명확히 정의

형식 (Numerical form)

$$V = (-1)^s \times M \times 2^E$$

- **sign bit** s: 양수(0) / 음수(1)
- **significand** M (= mantissa): 보통 $1.0 \leq M < 2.0$ 범위의 소수
  - "보통"인 이유: 0에 아주 가까운 수에서는 (이런 값이 denormalized) 이 범위를 벗어남
- **exponent** E: 2의 거듭제곱으로 값에 가중치 → 0 쪽으로 줄이거나 큰 수 쪽으로 늘림
- 인코딩: MSB가 s, 다음이 exp 필드 e (k비트), 마지막이 frac 필드 f (n비트)
  - e는 E를 그대로 저장하지 않음 (bias로 변환), f도 M을 그대로 저장하지 않음 (implied leading 1)



IEEE 754 필드 배치 (슬라이드 그림 재구성)

형식	전체	s	exp (e)	frac (f)	bias
Single (float)	32 bits	1	8	23	127
Double (double)	64 bits	1	11	52	1023

- single과 double은 메커니즘이 동일, 비트 수(정밀도)만 다름 → 수업은 single 위주
- **외울 것: 각 필드에 몇 비트씩 쓰는지**

Value Classes



NaN은 이 수직선 어디에도 없음 (비교 불가)

exp 필드 패턴으로 나뉘는 값의 분류와 수직선 배치 (슬라이드 그림 재구성). ±∞는 양·음 방향의 overflow.

Normalized

exp 필드가 00…0도 11…1도 아님 → $E = \text{Exp} - \text{Bias}$, $M = 1 + f$

Denormalized

exp 필드 00…0 → $E = 1 - \text{Bias}$, $M = f$

Special

exp 필드 11…1 → $f = 0$이면 $\pm\infty$, $f \neq 0$이면 NaN

---

normalized는 0 근처 아주 작은 수를 표현하기 어려움 → denormalized로 보완, 둘 사이를 **매끄럽게 연결**하는 것이 목표

3 Normalized Value

조건: `exp ≠ 00…0 이고 exp ≠ 11…1`

$$E = \text{Exp} - \text{Bias} \qquad \text{Bias} = 2^{k-1} - 1 \qquad M = 1 + f = 1.\text{frac}_2$$

- Exp: exp 필드를 unsigned로 읽은 값, k: exp 비트 수
- Single: k = 8, Bias = 127, Exp 1 ~ 254 → E −126 ~ 127 · Double: k = 11, Bias = 1023, Exp 1 ~ 2046 → E −1022 ~ 1023
- bias가 필요한 이유: exp를 그대로 쓰면 2^{-10} 같은 음의 지수를 표현할 수 없음 → bias를 unsigned 범위의 대략 가운데(254의 절반 ≈ 127)로 잡아 양·음 지수를 고르게 나눔
- **Implied leading 1**: $M = 1.\text{frac}$의 맨 앞 1은 저장하지 않음 → 비트 하나를 공짜로 얻음 (get extra leading bit for free)

  - frac = 00⋯0 → $M = 1.0$ (최소) · frac = 11⋯1 → $M = 2.0 - \epsilon$ (최대)

손계산 1 · Float 15213.0 인코딩

- $15213 = 11101101101101_2$. binary point를 왼쪽으로 13칸 옮김 → 값이 2^{13}으로 나뉜 셈 → $E = 13$으로 보정: $15213 = 1.1101101101101_2 \times 2^{13}$
- significand: $M = 1.1101101101101_2$, leading 1을 빼고 23비트로 채움 → frac = `110 1101 1011 0100 0000 0000`
- exponent: $E = 13, \mathrm{Exp} = 13 + 127 = 140 = 10001100_2$
- 비트 배치: s = 0 · exp = `1000 1100` · frac(23) = `110 1101 1011 0100 0000 0000` → **0x466DB400**

4 Denormalized Value

조건: exp = 00⋯0 $E = 1 - \mathrm{Bias} \qquad M = f = 0.\mathrm{frac}_2$

- single: $E = -126$ (가장 작은 normalized와 같은 지수), double: $E = -1022$
- leading 1을 붙이지 않음 → frac 비트를 그대로 $0.\mathrm{frac}$로 해석. exp가 all-0이면 컴퓨터가 "normalized 아님 → leading 1 붙이지 않음"으로 판단

왜 1 − Bias인가

가장 작은 normalized: $\mathrm{Exp} = 1 \Rightarrow E = 1 - \mathrm{Bias}, M = 1.0$. 이 값과 0 사이에는 implied leading 1 때문에 큰 틈이 있다. denorm도 $E = 1 - \mathrm{Bias}$를 쓰면 같은 지수 → 이웃 값 간격이 동일 → normalized로 매끄럽게 이어진다. ($E = -\mathrm{Bias}$를 쓰면 denorm 간격이 절반이 되어 경계에서 끊김)

- 두 가지 용도
  - **0 표현**: $e = 0, f = 0$ → 값 0. $+0$과 -0이 따로 존재 (sign bit만 다름). $1/+0 = +\infty$, $1/-0 = -\infty$처럼 부호 구분이 의미 있는 경우가 있음
  - **0에 아주 가까운 수**: $M = f < 1$, **gradual underflow**: 최소 normalized보다 작은 수를 0으로 바로 떨어뜨리지 않고 균등 간격으로 표현. 가장 큰 denorm = $(1 - \epsilon) \times 2^{-126}$ → 바로 다음이 최소 normalized

5 Special Value

- 조건: exp = 11…1
  - $f = 0$: $\pm\infty$: overflow 결과, 양·음 둘 다 있음. $1/0 = +\infty$, $-1/0 = -\infty$
  - $f \neq 0$: **NaN** (Not-a-Number): 수치 값을 정할 수 없는 경우: $\sqrt{-1}$, $\infty - \infty$, $\infty \times 0$

설명	exp	frac	값	Single	Double
Zero	00…00	00…00	0.0	0	0
Smallest pos. denorm	00…00	00…01	$2^{-23,-52} \times 2^{-126,-1022}$	$2^{-149} \approx 1.4 \times 10^{-45}$	$2^{-1074} \approx 4.9 \times 10^{-324}$
Largest denorm	00…00	11…11	$(1.0 - \epsilon) \times 2^{-126,-1022}$	$\approx 1.18 \times 10^{-38}$	$\approx 2.2 \times 10^{-308}$
Smallest pos. normalized	00…01	00…00	$1.0 \times 2^{-126,-1022}$	$\approx 1.18 \times 10^{-38}$	$\approx 2.2 \times 10^{-308}$
One	01…11	00…00	1.0	0x3F800000	0x3FF0000000000000
Largest normalized	11…10	11…11	$(2.0 - \epsilon) \times 2^{127,1023}$	$\approx 3.4 \times 10^{38}$	$\approx 1.8 \times 10^{308}$

- largest denorm과 smallest normalized는 바로 이웃 (차이 2^{-149}, single 기준)

6 Tiny Floating-Point 예제

8-bit 형식

- s 1비트 (MSB) · exp 4비트 (bias $2^{4-1} - 1 = 7$) · frac 3비트. IEEE와 같은 규칙 (normalized / denormalized / special). denorm도 $E = 1 - 7 = -6$, normalized의 최소도 $E = 1 - 7 = -6$

손계산 2 · Dynamic Range (양수만)

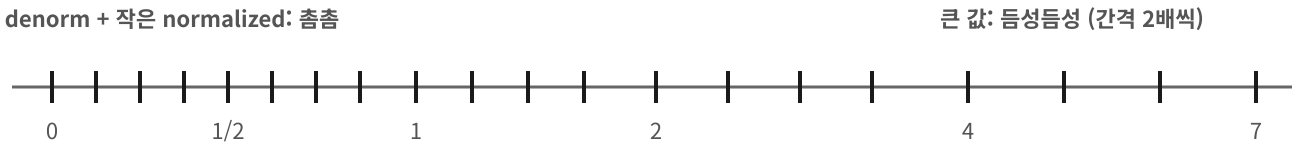
s	exp	frac	E	값	비고
0	0000	000	−6	0	zero
0	0000	001	−6	1/8 × 1/64 = 1/512	closest to zero
0	0000	110	−6	6/512	denorm
0	0000	111	−6	7/512	largest denorm
0	0001	000	−6	8/8 × 1/64 = 8/512	smallest norm
0	0001	001	−6	9/512	norm
0	0110	110	−1	14/8 × 1/2 = 14/16	
0	0110	111	−1	15/16	closest to 1 below
0	0111	000	0	1	exp = bias
0	0111	001	0	9/8	closest to 1 above
0	1110	110	7	14/8 × 128 = 224	
0	1110	111	7	15/8 × 128 = 240	largest norm
0	1111	000	없음	∞	

- 값 = (frac에서 온 인수) × (exp에서 온 인수 2^E)
- denorm → 첫 normalized: 간격이 $1/512$로 그대로 유지 → **smooth transition**. normalized $8/512$에는 implicit leading 1이 있지만 E가 같아서 $8/512 = 7/512 + 1/512$로 이어짐
- 0 ~ 1 사이에 비트 조합의 거의 절반이 몰려 있음 (exp가 bias보다 작은 모든 조합)
- exp가 커질수록 이웃 값 간격이 커짐: $E = -6$에서 2^{-9}, $E = 7$에서 2^4

6-bit 형식: 값의 분포

- s 1 · exp 3 (bias 3) · frac 2. exp별 표현되는 양수 값 (간격):

exp	E	값들	간격
000 (denorm)	−2	0, 1/16, 2/16, 3/16	1/16
001	−2	4/16, 5/16, 6/16, 7/16	1/16
010	−1	1/2, 5/8, 3/4, 7/8	1/8
011	0	1, 5/4, 3/2, 7/4	1/4
100	1	2, 5/2, 3, 7/2	1/2
101	2	4, 5, 6, 7	1
110	3	8, 10, 12, 14	2
111	없음	∞ (f=00), NaN	없음



6-bit tiny FP의 양수 값 분포: 0에 가까울수록 촘촘, exp가 1 늘 때마다 간격 2배 (슬라이드 그림 재구성)

- 각 exp 구간마다 값이 4개씩 (frac 2비트), 수직선 −14 ~ 14 사이에 분포
- **gradual underflow (close-up)**: denorm(0 ~ 3/16)의 간격 1/16이 바로 다음 normalized 구간 (4/16 ~ 7/16)의 간격과 같음

Encoding의 특별한 성질

- FP zero = integer zero: 모든 비트가 0
- 비트 패턴을 1씩 증가시키면 (s 고정) 값이 0 → 최대 → ∞까지 **단조 증가**
  - frac이 all-1에서 넘치면 exp가 1 증가 → 값이 2배 되는 효과 → 계속 커짐

  - → (거의) unsigned integer 비교로 대소 비교 가능

- 예외 처리 필요: sign bit를 먼저 비교, $-0 = +0$으로 취급, NaN (비트상 어떤 값보다 크게 나옴 → 비교 결과를 무엇으로 할지 문제). 그 외 (denorm vs normalized, normalized vs ∞)는 그대로 OK

IEEE 754 binary32 — 부동소수점 실험실

프리셋 — Presets

영점과 정수: +0 | −0 | 1.0

정규 / 비정규 경계: 최소 비정규 | 최대 비정규 | 최소 정규 | 최대 유한값

오차 / 특수값: 0.1 | +∞ | NaN

10진수 인코딩 — Decimal Encoder

10진수 / 특수값: 0.1

0.1 최근접 짝수 반올림 후 인코드

32비트 필드 — Sign / Exponent / Fraction

부호(Sign) · 1bit

0

비트 31 → 31 · 하나의 연속된 1비트 필드

지수(Exponent) · 8bit

0	1	1	1	1	1	1	1

비트 30 → 23 · 하나의 연속된 8비트 필드

가수(Fraction) · 23bit

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

비트 22 → 0 · 하나의 연속된 23비트 필드

```
비트 패턴: 0x3F800000
분류: 정규(normalized)
근삿값(JS Number 표시): 1
```

디코딩 공식 — IEEE 754

$s=0,\ e=127,\ frac=0$

$E = 127 - 127 = 0$

$M = 1 + 0 / 2^{23}$

$값 = (-1)^{0} \times M \times 2^{0}$

> 정규수(normalized)는 숨은 선행 1을 사용합니다. 비정규수(subnormal)는 숨은 선행 1이 없습니다.

정확한 값 — Exact Representation

▼ 정확한 유리수 / 10진수 전개 펼치기

```
유리수(num/den)
1/1

정확한 10진수
1
```

7 Rounding

연산의 기본 아이디어

$$\mathrm{Round}(x \ \mathrm{op}\ y)$$

- ① 먼저 **정확한 결과**를 계산 (연산기 내부는 더 많은 비트로 계산 가능) → ② 원하는 precision에 맞춤
  - exponent가 너무 크면 overflow → $\pm\infty$ · frac에 맞도록 round

Rounding Mode

모드	$1.40	$1.60	$1.50	$2.50	−$1.50
Towards zero	$1	$1	$1	$2	−$1
Round down ($-\infty$)	$1	$1	$1	$2	−$2
Round up ($+\infty$)	$2	$2	$2	$3	−$1
Nearest even (default)	**$1**	**$2**	**$2**	**$2**	**−$2**

Round-to-Even이 기본인 이유

나머지 모드는 통계적으로 biased: 양수들의 합이 항상 과대 또는 과소 추정됨. nearest는 버리는 부분이 절반보다 작으면 내림, 크면 올림 → 균등 분포라면 50:50으로 공평. 문제는 **정확히 절반**일 때 → 결과의 최하위 자리가 **짝수**가 되도록 → 절반은 올림, 절반은 내림 → 편향 없음.

- 1.2349999 → 1.23 (절반 미만) · 1.2350001 → 1.24 (절반 초과) · 1.2350000 → 1.24 (정확히 절반, 3은 홀수 → 올림) · 1.2450000 → 1.24 (정확히 절반, 4는 짝수 → 내림)

Binary Rounding

- even: LSB가 0 · halfway: rounding 위치 오른쪽 비트들이 `100…0`
  - 버리는 첫 비트가 0 → 절반 미만 → 버림
  - 버리는 첫 비트가 1이고 뒤에 1이 하나라도 → 절반 초과 → 올림
  - 버리는 첫 비트가 1이고 뒤가 전부 0 → 정확히 절반 → even 쪽으로

이진 반올림 — Rounding Modes

남길 소수 비트(k) 2 이진 소수 1.101

Halfway: 아래쪽 짝수 | Halfway: 위쪽 짝수 | 음수 Halfway | 2.875 → 3 | Halfway 아님

입력값과 양자화 간격

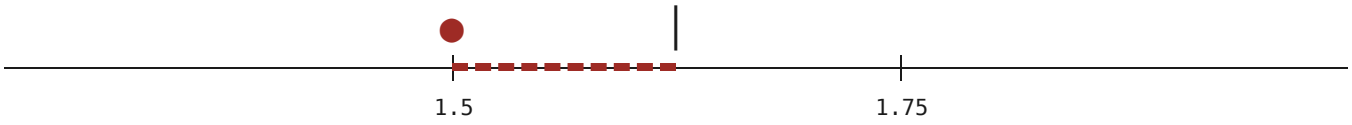
1.101 (binary) = 1.625 (decimal)

남길 간격 $\Delta = 1 / 2^2 = 0.25$

반올림 결과 — 4개 모드

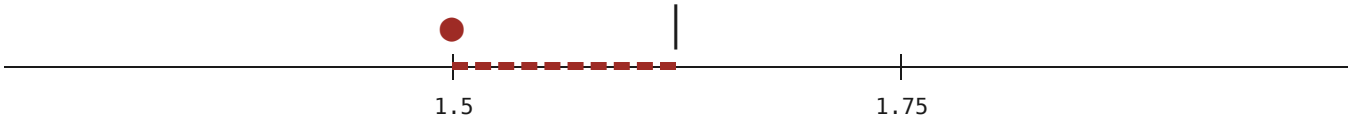
최근접 짝수 — Round to Nearest, Ties to Even

결과: 1.5 · 이진: 1.10



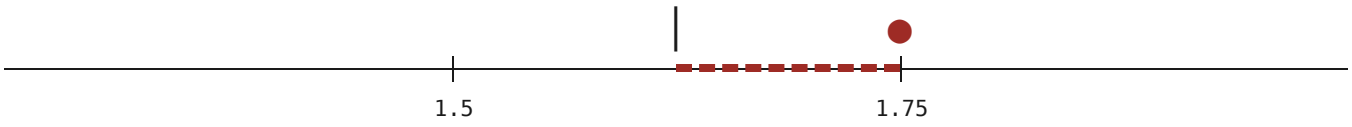
0 방향 절삭 — Toward Zero

결과: 1.5 · 이진: 1.10



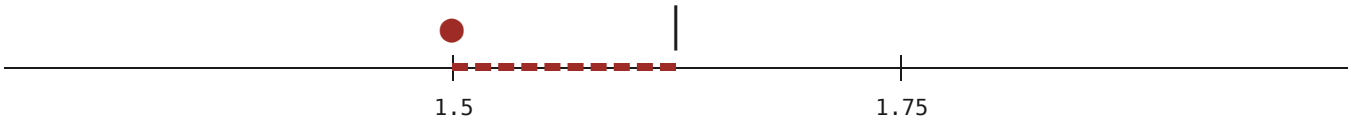
양의 무한대 방향 올림 — Toward +∞

결과: 1.75 · 이진: 1.11



음의 무한대 방향 내림 — Toward −∞

결과: 1.5 · 이진: 1.10



숫자선은 각 표시 구간 안에서의 상대 위치를 나타냅니다.
|는 원래 입력, 빨간 ●는 반올림 결과, 빨간 점선은 두 값 사이의 거리입니다.
◆는 입력과 결과가 완전히 일치하는 위치입니다.

정확한 halfway(중간값)입니다.
최근접 짝수 반올림은 두 후보 중 반올림 후 남기는 마지막 비트가 0인 값(정수 배율에서 짝수)을 선택합니다.
선택한 값: 1.5.

0 방향 절삭은 부호와 무관하게 0에 가까운 쪽을 선택합니다.
음의 무한대 방향 내림은 항상 작거나 같은 격자점을 선택합니다.
최근접 짝수 규칙은 halfway에서 특정 방향을 반복적으로 선택하는 편향을 줄입니다.

손계산 3 · 1/4 단위(소수점 아래 2비트) Rounding

값	이진	버리는 부분	판정	결과	결과 값
2 3/32	10.00**011**	011	1/2 미만 → 내림	10.00	2
2 3/16	10.00**110**	110	1/2 초과 → 올림	10.01	2 1/4
2 7/8	10.11**100**	100	= 1/2, LSB 1(홀) → 올림	11.00	3
2 5/8	10.10**100**	100	= 1/2, LSB 0(짝) → 내림	10.10	2 1/2

슬라이드 Action 열은 네 줄 모두 "1/2 미만"으로 적혀 있는데, 위처럼 미만·초과·절반·절반이 맞음.

8 FP 곱셈과 덧셈

Multiplication

$$(-1)^{s_1} M_1 \, 2^{E_1} \;\times\; (-1)^{s_2} M_2 \, 2^{E_2} \;=\; (-1)^{s} M \, 2^{E}$$

- sign s: $s_1 \oplus s_2$ (XOR) → 같으면 양수, 다르면 음수 · significand M: $M_1 \times M_2$ · exponent E: $E_1 + E_2$ (지수는 덧셈만)
- Adjustment: $M \geq 2$ → M을 오른쪽 shift, E 1 증가 (leading 1이 binary point 바로 왼쪽에 하나만 오도록) · E가 범위 밖 → overflow · M을 precision에 맞게 round
- 가수의 곱 자체는 정규화 유지 — 지수 합이 표현 범위 아래로 내려가면 결과는 denormalized(또는 0)가 될 수 있음: $M_1, M_2 \geq 1 \Rightarrow M_1 M_2 \geq 1$ · 구현에서 가장 큰 일: significand끼리의 곱

Addition

- 가정 $E_1 \geq E_2$: binary point 정렬: M_2를 $E_1 - E_2$만큼 오른쪽 shift → 같은 지수로 맞춤 (10진수 덧셈에서 자릿수 맞추는 것과 같음)
- 정확한 결과: sign·M = signed align & add 결과, $E = E_1$
- Adjustment: $M \geq 2$ → 오른쪽 shift, E 증가 · $M < 1$ → 왼쪽 shift, E 감소 (부호가 다른 수를 더하면 생김) · E 범위 밖 → overflow · round
- 정규화하다가 E가 $1 - \mathrm{Bias}$보다 작아지면 → **denormalized로 저장** (leading 1 포기, $E = 1 - \mathrm{Bias}$로 고정하고 그만큼 shift)

수학적 성질

성질	덧셈 (Abelian group과 비교)	곱셈 (Commutative ring과 비교)
Closed	Yes (∞, NaN 가능)	Yes (∞, NaN 가능)
Commutative	Yes	Yes

성질	덧셈 (Abelian group과 비교)	곱셈 (Commutative ring과 비교)
Associative	**No** (overflow, rounding)	**No** (overflow, rounding)
Identity	0	1
Inverse / Distributes	Yes(∞, NaN 제외)	분배법칙 **No**
Monotonicity $a \geq b \Rightarrow a+c \geq b+c$	Yes (∞·NaN 제외)	Yes (∞, NaN 제외)

반례 (float 기준): 시험 단골

$$(3.14 + 1\text{e}10) - 1\text{e}10 = 0 \qquad 3.14 + (1\text{e}10 - 1\text{e}10) = 3.14$$

- $1\text{e}10$에 맞춰 3.14를 오른쪽 shift → significand 범위를 벗어나 사라짐 → 결과 0 (numpy float32로 확인: 0.0 / 3.14)

$$1\text{e}20 \times (1\text{e}20 - 1\text{e}20) = 0 \qquad 1\text{e}20 \times 1\text{e}20 - 1\text{e}20 \times 1\text{e}20 = \text{NaN}$$

- $1\text{e}20 \times 1\text{e}20$은 float 최대값($\approx 3.4 \times 10^{38}$) 초과 → $\infty - \infty$ = NaN (numpy float32로 확인: 0.0 / nan)

9 Floating Point 만들기 (Creating FP Number)

세 단계

1. **Normalize**: leading 1이 오도록 binary point 이동 ($1.xxx$ 꼴), 왼쪽으로 옮긴 만큼 exponent 증가
2. **Round**: frac 비트 수에 맞게 반올림
3. **Post-normalize**: rounding으로 $M = 10.0_2$이 되면 오른쪽 shift 1, exponent 1 증가

Guard · Round · Sticky 규칙

Guard bit G

결과에 남는 마지막 비트

Round bit R

버려지는 첫 비트

Sticky bit S

R 뒤 나머지 비트 전체의 OR

R = 0

나머지 0.5 미만 → 버림

R = 1, S = 1

나머지 0.5 초과 → G 자리에 1 더함 (올림)

R = 1, S = 0

나머지 = 0.5 → round to even: G = 1이면 올림, G = 0이면 버림

손계산 4 · 8-bit Unsigned → Tiny Float (s 1 · exp 4 · frac 3)

값	이진	① Normalize	E	G R S	② 올림?	Rounded	③ Post-norm	최종
128	10000000	1.0000000	7	0 0 0	N	1.000	없음	128
13	00001101	1.1010000	3	1 0 0	N	1.101	없음	13
17	00010001	1.0001000	4	0 1 0	N (halfway, G 짝)	1.000	없음	16
19	00010011	1.0011000	4	1 1 0	Y (halfway, G 홀)	1.010	없음	20
138	10001010	1.0001010	7	0 1 1	Y (1/2 초과)	1.001	없음	144
63	00111111	1.1111100	5	1 1 1	Y (1/2 초과)	10.000	1.000, E=6	64

- 63: 올림 결과 $M = 10.000_2$ → leading 1이 둘째 자리로 넘침 → 오른쪽 shift 1, $E = 5 \to 6$ → $1.000 \times 2^6 = 64$
- 슬라이드 표기 오류로 보이는 곳: "15 = 00001101" → 00001101은 13 (원래 예제는 13) · "19 = 10010011" → 19 = 00010011 (10010011은 147) · 138의 G R S "010" → 실제 0 1 1 (R 뒤에 1이 있으므로 S = 1, 그래서 올림 Y) · 138 최종 "142" → $1.001_2 \times 2^7 = 144$

10 C에서의 Floating Point

Casting

C의 두 precision: float (single), double (double). casting은 비트 표현을 바꿈 (값을 보존하려고 재인코딩)

double/float → int
소수부 truncate (round toward zero). 범위 밖·NaN은 정의되지 않음 → 보통 TMin
int → double

정확히 변환 (int가 53비트 이하이면) → 32비트 int는 frac 52비트 + leading 1에 다 들어감

int → float

frac이 23비트뿐 → **rounding mode에 따라 반올림될 수 있음**

- int → FP 변환 = 최상위 1이 leading 1이 되도록 오른쪽 shift, 그 뒤 비트가 frac. 32비트 int는 최대 31번 shift → double의 52비트 frac에는 들어가지만 float의 23비트에는 안 들어감

손계산 5 · int → Float 손실

- $16777217 = 2^{24} + 1$ → frac 23비트 뒤에 1이 하나 남음 → R = 1, S = 0, G = 0 → halfway, 짝수 쪽으로 버림
- (int)(float)x = 16777216 ≠ x · (int)(double)x는 그대로 16777217

Floating Point Puzzle

명제 (int x; float f; double d; d, f는 NaN 아님)	판정	이유
`x == (int)(float) x`	거짓	float frac 23비트 → 큰 int는 rounding (예: 16777217)
`x == (int)(double) x`	참	32비트 int는 double에 정확히 들어감
`f == (float)(double) f`	참	작은 표현 → 큰 표현 → 다시 작은 표현, 정보 손실 없음
`d == (float) d`	거짓	double → float에서 정밀도·범위 손실
`f == -(-f)`	참	부호 반전은 sign bit만 뒤집음
`2/3 == 2/3.0`	거짓	2/3은 정수 나눗셈 = 0, 2/3.0은 0.666…
`d < 0.0 ⇒ ((d*2) < 0.0)`	참	음수×2는 더 작은 음수, overflow해도 $-\infty$
`d > f ⇒ -f > -d`	참	부호만 뒤집으므로 대소 반전이 정확함
`d * d >= 0.0`	참	같은 부호 곱, overflow해도 $+\infty$
`(d + f) - d == f`	거짓	결합법칙 불성립 (예: d가 매우 크면 f가 rounding으로 사라짐)

11 요약과 시험 포인트

- IEEE FP는 명확한 수학적 성질을 가짐: $x/2^k$ 꼴의 수를 표현. 연산은 구현과 무관하게 추론 가능 → 완벽한 정밀도로 계산한 뒤 round한 것처럼
- 실수 연산과 같지 않음 → 결합법칙·분배법칙 위반 → 컴파일러, 수치 소프트웨어 작성이 어려워짐

- fractional binary: $x/2^k$ 꼴만 정확히 표현, 1/3 · 1/5 · 1/10은 반복 비트
- 필드 비트 수: single 1/8/23 (bias 127)
- double 1/11/52 (bias 1023)
- normalized: $E = \mathrm{Exp} - \mathrm{Bias}$, $M = 1 + f$ / denorm: $E = 1 - \mathrm{Bias}$, $M = f$ / exp all-1: $\pm\infty$ 또는 NaN
- denorm이 $1 - \mathrm{Bias}$를 쓰는 이유: 최소 normalized와 같은 간격 → gradual underflow, smooth transition
- $+0$과 -0 구분, NaN 예 ($\sqrt{-1}$, $\infty - \infty$, $\infty \times 0$)
- 15213.0 → 0x466DB400 같은 인코딩 / 디코딩 손계산
- tiny 8-bit (bias 7) 표: 최소 denorm 1/512, 최대 denorm 7/512, 최소 norm 8/512, 최대 norm 240
- 값 분포: 0 근처 촘촘, exp 1 증가마다 간격 2배
- 비트 패턴이 단조 → unsigned 비교로 대소 비교 가능 (부호, ±0, NaN 예외)
- rounding 4모드 표, round-to-even이 기본인 이유 (통계적 편향 없음)
- 이진 반올림: G · R · S 규칙, halfway면 LSB 짝수로
- FP 곱셈: $s = s_1 \oplus s_2$, $M = M_1 \times M_2$, $E = E_1 + E_2$
- 덧셈: 지수 맞춰 shift 후 더하고 정규화
- 결합법칙·분배법칙 불성립 반례 (3.14 + 1e10 − 1e10, 1e20 × 1e20 − …)
- 변환 3단계 normalize → round → post-normalize (63 → 64 사례)
- C casting: FP → int는 truncate, int → double 정확, int → float 반올림 가능, puzzle 10개

12 연습문제

3-1IEEE single 인코딩★★

-12.375를 IEEE single precision으로 인코딩하라 (s, exp, frac, hex).

힌트 보기

---

normalize → $1.xxx \times 2^E$ 꼴로 만들고, Exp = E + 127, frac은 leading 1을 뺀 뒷부분.

정답 보기

개념 정리는 힌트에 있음.

① 이진수로: $12.375 = 1100.011_2$.

> 8+4, 0.25+0.125

② Normalize: $1.100011_2 \times 2^3$.

> point를 3칸 왼쪽으로

③ 필드 채우기: s = 1, Exp = 3 + 127 = 130 = `10000010`, frac = `10001100000000000000000`.

④ 합쳐서 hex로: `1 10000010 10001100…`.

최종 답 **0xC1460000 (s=1, exp=`10000010`, frac=`1000110…0`)**

3-2IEEE single 디코딩★★

single precision 비트 패턴 `0xC0A00000` 이 나타내는 값을 구하라.

힌트 보기

s | exp(8) | frac(23)로 자른 뒤, E = Exp − 127, M = 1.frac.

정답 보기

① 비트로 펼친다: `1100 0000 1010 0000 ⋯` → s = 1, exp = `10000001` = 129, frac = `0100⋯0` .

② 지수와 가수: E = 129 − 127 = 2, M = $1.01_2 = 1.25$.

③ 값을 계산한다: $V = -1 \times 1.25 \times 2^2$.

최종 답 $V = -1.25 \times 4 = \mathbf{-5.0}$

3-38-bit tiny format★★

8-bit tiny format (s 1, exp 4, bias 7, frac 3)에서 ① 0.75의 비트 패턴 ② `0 0000 101`의 값 ③ `0 1000 011`의 값을 구하라.

힌트 보기

exp = 0000이면 denorm ($E = -6, M = f$), 아니면 normalized ($E = \mathrm{Exp} - 7, M = 1 + f$).

정답 보기

① $0.75 = 1.1_2 \times 2^{-1}$ → Exp = −1 + 7 = 6 = `0110`, frac = `100`.

> normalized

② exp = 0000 → denorm: $M = f = 5/8, E = -6 \to 5/8 \times 2^{-6}$.

> leading 1 없음

③ Exp = 8 → $E = 1, M = 1 + 3/8 = 11/8 \to 11/8 \times 2$.

최종 답 ① **0 0110 100** ② $5/512$ ③ $11/4 = 2.75$

3-4Round-to-even 손계산★★

round-to-even으로 소수점 아래 2비트(1/4 단위)까지 반올림하라: ① 10.011_2 (2.375) ② 11.101_2 (3.625) ③ 1.00101_2 (1.15625)

힌트 보기

버리는 첫 비트 = R, 그 뒤 전부 OR = S. R=1·S=0이면 정확히 절반 → 짝수 쪽.

정답 보기

① ① 버리는 부분 `1` = halfway, 남는 LSB 1 → 올림 → $10.10_2 = 2.5$.

② ② 버리는 부분 `1` = halfway, LSB 0 → 내림 → $11.10_2 = 3.5$.

③ ③ 버리는 부분 `101` > halfway → 올림 → $1.01_2 = 1.25$.

최종 답 **① 2.5 ② 3.5 ③ 1.25**

3-5정수 → Tiny Float 변환★★★

8-bit unsigned 25와 27을 tiny format(frac 3비트)으로 변환하라. normalize · G R S · 최종 값을 쓰라.

힌트 보기

손계산 4 표의 절차 그대로: normalize → frac 3비트에 맞춰 G R S 판정 → 필요하면 post-normalize.

정답 보기

① 25 = `00011001` → $1.1001_2 \times 2^4$ → frac 3비트 기준 `1.100 | 1` → G = 0, R = 1, S = 0.

> halfway, G 짝

② → 버림 → $1.100_2 \times 2^4 = 24$.

③ 27 = `00011011` → $1.1011_2 \times 2^4$ → `1.101 | 1` → G = 1, R = 1, S = 0.

> halfway, G 홀

④ → 올림 → $1.110_2 \times 2^4 = 28$.

최종 답 **25 → 24 (버림), 27 → 28 (올림)**

3-6int → float/double 왕복★★

`int x = 16777217;` 일 때 `x == (int)(float)x` 와 `x == (int)(double)x` 의 결과와 이유를 쓰라.

힌트 보기

$16777217 = 2^{24} + 1$: leading 1 뒤에 24비트가 필요하다. float frac는 23비트, double frac는 52비트.

정답 보기

① $16777217 = 2^{24} + 1$: leading 1 뒤에 24비트가 필요.

② float frac는 23비트 → 마지막 1이 버려짐 (halfway, 짝수로) → 16777216 → **거짓**.

③ double frac는 52비트 → 정확히 표현 → **참**.

최종 답 **(int)(float)x: 거짓 (16777216), (int)(double)x: 참**

3-7곱셈 결합법칙 반례★★

float에서 `(1e20 * 1e20) * 1e-20`과 `1e20 * (1e20 * 1e-20)`의 결과가 다른 이유를 설명하라.

힌트 보기

float의 최대 normalized는 $\approx 3.4 \times 10^{38}$. overflow는 $+\infty$가 된다.

정답 보기

① 앞 식: $1e20 \times 1e20 = 10^{40}$이 float 최대값 3.4×10^{38}을 넘음 → $+\infty$ → $+\infty \times 10^{-20} = +\infty$.

> 먼저 곱해서 overflow

② 뒤 식: $1e20 \times 1e{-}20 = 1 \rightarrow 1e20 \times 1 = 1e20$.

> 지수 상쇄가 먼저

최종 답 **앞 = $+\infty$, 뒤 = $1e20$. overflow 때문에 곱셈 결합법칙이 성립하지 않는다**